

SZABIST
UNIVERSITY
Islamabad Campus

AI-Based Autonomous Cyber Defense System

(Mid-Term Report)

Muhammad Bilal Khan (23108222)
Nair Abbas (23108228)
Hammad Qureshi (23108207)

Supervised by: Muhammad Hasaan Mujtaba

In partial fulfillment of requirement for the degree
Bachelor of Science (Artificial Intelligence)

Department of Robotics & Artificial Intelligence
Faculty of Computing and Engineering Sciences
SZABIST University
Islamabad, Pakistan

Fall 2026

AI-Based Autonomous Cyber Defense System

By

Muhammad Bilal Khan (23108222)
Nair Abbas (23108228)
Hammad Qureshi (23108207)

A PROJECT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF BACHELOR OF SCIENCE (ARTIFICIAL INTELLIGENCE)

(Supervisor) Muhammad Hasaan Mujtaba	(FYP Committee Head) Mr. Hamza Imran

(Program Manager BS(AI))
Mr. Hamza Imran

Department of Robotics & Artificial Intelligence
Faculty of Computing and Engineering Sciences
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology University
Islamabad, Pakistan

Fall 2026

DECLARATION

We, the candidates of Bachelor of Science (Artificial Intelligence) at Shaheed Zulfikar Ali Bhutto Institute of Science and Technology University, Islamabad do hereby certify that this report titled **AI-Based Autonomous Cyber Defense System**, submitted as partial fulfillment of Bachelor of Science (Artificial Intelligence) degree requirements, is our original work and we are its sole authors. All the employed materials, references to the literature, and the work of others have been referred to and duly cited. This report has not been presented for examination anywhere else.

Muhammad Bilal Khan	Nair Abbas	Hammad Qureshi
23108222	23108228	23108207

Project Overview

Security teams in modern organisations have to make sense of very large volumes of network traffic, alerts and suspicious files. Much of this work is still manual and spread across separate tools, so a single incident can require an analyst to check network monitors, malware scanners and log viewers one after another before the picture becomes clear. When attacks are frequent, or when the activity does not match any known signature, this way of working is slow and error-prone.

The *AI-Based Autonomous Cyber Defense System* proposes a modular platform in which machine learning components analyse different kinds of security evidence and report their findings in a common, structured form. At the mid-term stage, three analysis modules have been implemented and evaluated as working prototypes on public benchmark datasets. The **Network Threat Classification** module uses a two-stage LightGBM cascade on CIC-IDS2017 flow records to decide whether a flow is benign and, if not, which attack family it belongs to. The **Malware Classification** module uses CatBoost to separate benign and malicious Windows executables represented by BODMAS feature vectors, followed by a Random Forest that assigns the malware to one of eight categories. The **Anomaly Detection** module learns only from normal traffic, using an ensemble of unsupervised detectors (Isolation Forest, Local Outlier Factor and an autoencoder) so that it can flag behaviour that has never been seen in training. A log analysis component is under development, while the remaining proposed parts (network monitoring, incident correlation, vulnerability and attack-path analysis, attack prediction, risk analysis, an AI investigation agent and a unified dashboard) are planned for the second half of the project.

The scope of the project is a software prototype developed and tested in a controlled environment with public datasets. It is intended to support, not replace, security analysts. It does not scan production networks, execute countermeasures automatically or develop malware. The modules currently run as separate prototypes with their own command-line or Gradio interfaces; connecting them through a shared backend and dashboard is the main integration goal for the next phase.

The expected impact is a measurable reduction in the manual effort needed to triage security events. Results from the three modules are encouraging. The network classifier reached 99.91% holdout accuracy with a macro F1-score of 0.958, the malware detector 99.51% accuracy, and the improved anomaly detector raised its F1-score on deduplicated CIC-IDS2017 traffic from 0.07 to 0.83 while detecting 85% of attack types never seen during training on NSL-KDD. These results come from benchmark data and still need validation on live traffic, which is part of the future work.

Dedication

We dedicate this work to Almighty Allah, whose blessings gave us the health, patience and opportunity to pursue our studies.

We also dedicate it to our parents and families, whose love, sacrifices and constant encouragement have supported us at every stage of our education, and to our teachers, who taught us to ask careful questions and to value honest work.

Acknowledgment

We are grateful to our supervisor, Muhammad Hasaan Mujtaba, for his guidance, feedback and patience throughout the first half of this project. His comments during our review meetings helped us narrow the scope of the system and keep our evaluation honest.

We thank the Department of Robotics & Artificial Intelligence and the Faculty of Computing and Engineering Sciences at SZABIST University, Islamabad, for the academic environment and resources that made this work possible. We also acknowledge the researchers who made the CIC-IDS2017, NSL-KDD and BODMAS datasets publicly available; without these resources the evaluation reported here could not have been carried out.

Finally, we thank each other for the cooperation within the team, and our families and friends for their support and understanding during long periods of work.

Revision History

Compiled By	Checked By	Date	Description	Version
Muhammad Bilal Khan	Muhammad Hasaan Mujtaba	01-Oct-2026	Initial Proposal	1.0
Nair Abbas	Muhammad Hasaan Mujtaba	03-Oct-2026	Project Description	1.1
Hammad Qureshi	Muhammad Hasaan Mujtaba	05-Oct-2026	Project Modules	1.2
Team	Muhammad Hasaan Mujtaba	07-Oct-2026	Final Proposal Draft	1.3
Team	Muhammad Hasaan Mujtaba	Nov-2026	Chapter 1: Introduction	2.0
Team	Muhammad Hasaan Mujtaba	Nov-2026	Chapter 2: Background and Literature Review	2.1
Team	Muhammad Hasaan Mujtaba	Nov-2026	Chapter 3: Software Requirements Specification	2.2
Team	Muhammad Hasaan Mujtaba	Nov-2026	Mid-Term Report (Chapters 1–3)	2.3

Contents

Project Overview ii

Dedication iii

Acknowledgment iv

Revision History v

List of Figures viii

List of Tables ix

1 Introduction 1
- 1.1 Product Purpose . . . 2
- 1.2 Product Scope . . . 2
- 1.3 Objectives . . . 4
- 1.4 Intended Market of Product . . . 5
- 1.5 Intended Users of the Product . . . 6

2 Background and Literature Review 7
- 2.1 Existing System Description . . . 7
  - 2.1.1 Related Academic Work . . . 7
  - 2.1.2 Review of Existing Systems . . . 8
- 2.2 Future System Usage Analysis . . . 12
- 2.3 Problem Statement / Limitations . . . 13
- 2.4 Proposed Solution . . . 15
  - 2.4.1 System Architecture and Workflow . . . 15
  - 2.4.2 Module 1: Network Threat Classification . . . 16
  - 2.4.3 Module 2: Malware Classification . . . 18
  - 2.4.4 Module 3: Anomaly Detection . . . 20
  - 2.4.5 Relationship Between the Modules . . . 22
  - 2.4.6 Planned Components . . . 23
  - 2.4.7 Design Considerations . . . 23
- 2.5 Software Process Model . . . 24

3 Software Requirements Specification 26
- 3.1 Introduction . . . 26
  - 3.1.1 Document Scope . . . 26
  - 3.1.2 Audience . . . 26
- 3.2 Functional Requirements . . . 27
- 3.3 Non-Functional Requirements . . . 29
  - 3.3.1 Software Quality Attributes . . . 29
  - 3.3.2 Performance Requirements . . . 30
  - 3.3.3 Safety Requirements . . . 30

3.3.4 Other Non-Functional Requirements 30
3.4 Requirements Gathering Techniques Used 31
3.4.1 Literature and Existing System Review 31
3.4.2 Supervisor Meetings 31
3.4.3 Dataset and Document Analysis 31
3.5 Time Frame 32

References **33**

List of Figures

2.1 Overall architecture of the AI-Based Autonomous Cyber Defense System and implementation status at mid-term 16
2.2 Two-stage workflow of the Network Threat Classification module 17
2.3 Confusion matrices of the Network Threat Classification module on the holdout set: binary gate (left), Stage 2 on true attacks (centre) and end-to-end cascade (right) 18
2.4 Two-stage workflow of the Malware Classification module 19
2.5 Workflow of the Anomaly Detection module 20
2.6 Anomaly Detection on CIC-IDS2017: original vs. improved vs. hybrid (test set, mean ± standard deviation over three seeds) 22
2.7 Incremental development model applied to the project, with the status of each increment at mid-term 25

List of Tables

2.1 Applications Comparison 12
2.2 Class distribution of the prepared CIC-IDS2017 data used by the Network Threat Classification module 16
2.3 Holdout performance of the Network Threat Classification module (CIC-IDS2017) 17
2.4 Malware categories used by the second stage of the Malware Classification module (BODMAS) 18
2.5 Test results of the Malware Classification module (as reported in the module documentation) 19
2.6 Configuration of the improved (v2) anomaly detector selected on validation data 21
2.7 Anomaly Detection test results: original (v1), improved (v2) and hybrid (mean of three seeds) 22
2.8 Implementation status of the proposed components at mid-term 23

3.1 Functional Requirements 27
3.2 Time Frame 32

Chapter 1

Introduction

Almost every organisation now depends on networked systems for its daily work. Servers, workstations, cloud services and applications exchange data continuously, and each of these components can be targeted by attackers. Defending such an environment means watching several kinds of evidence at once: the traffic moving across the network, the files that arrive on endpoints, and the events recorded in system and application logs. Guidance on incident handling treats detection and analysis as a core phase of incident response, and points out that it depends on collecting and interpreting evidence from many different sources [1].

In practice this evidence is often examined with separate tools. A network intrusion detection system raises alerts about suspicious flows, an antivirus or endpoint product reports suspicious files, and a log management platform stores events for later search. Commercial security information and event management (SIEM) and extended detection and response (XDR) products such as Splunk Enterprise Security, Microsoft Sentinel and CrowdStrike Falcon bring much of this information together [2], [3], [4], and open-source platforms such as Wazuh offer comparable monitoring features [5]. Even so, an analyst must still decide which alerts matter, recognise what kind of attack they represent and judge whether unfamiliar behaviour is harmful. When alerts are numerous, or when an attack does not match an existing rule, this work becomes slow and depends heavily on individual expertise.

Machine learning has been studied for many years as a way to support these decisions. Supervised classifiers can learn the patterns of known attack families from labelled data [6], while anomaly detection methods learn what normal behaviour looks like and flag deviations from it [7]. Each approach has well-known weaknesses. Supervised models are usually accurate on the attacks they were trained on but cannot recognise genuinely new ones, and anomaly detectors tend to produce more false alarms and are harder to interpret [8]. Results obtained on benchmark datasets also do not always transfer to real networks [8], [9]. These observations suggest that no single model is sufficient, and that different detectors should be combined in a way that keeps their outputs understandable to a human analyst.

This project, the *AI-Based Autonomous Cyber Defense System*, takes that view. It is being developed as a modular platform in which separate machine learning components examine different kinds of security evidence: network flows, malware feature vectors and deviations from normal traffic. Each component returns a structured result that can later be combined with the others. The long-term plan, set out in the project proposal, also includes log analysis, incident correlation, vulnerability and attack-path analysis, risk estimation, an AI investigation agent and a dashboard. The word “autonomous” in the title refers to automating analysis and the preparation of recommended responses. The system is designed to assist analysts, not to take enforcement actions on production systems by itself.

This mid-term report covers the first half of the project. By this stage, three analysis modules have been implemented and evaluated on public datasets: network threat classification, malware classification and anomaly detection. Log analysis is under development, and the remaining modules and the integration layer are planned. Throughout the report, implemented functionality is clearly separated from planned functionality. The rest of this chapter describes the purpose, scope, objectives, intended market and intended users of the system.

1.1 Product Purpose

The purpose of the system is to reduce the manual effort needed to recognise and understand security threats by applying machine learning to several kinds of security data and presenting the results in a consistent, explainable form. Three needs drive the design.

The first is to **detect and name known threats quickly**. When a network flow belongs to a familiar attack family, such as a denial-of-service flood or a port scan, the analyst benefits from an immediate and specific label instead of a generic alert. The Network Threat Classification module serves this need for network traffic, and the Malware Classification module serves it for Windows executables represented as feature vectors.

The second is to **notice behaviour that no rule or classifier has seen before**. Supervised models can only recognise the categories they were trained on. The Anomaly Detection module therefore learns a profile of normal traffic and reports flows that deviate from it, together with the features that made each flow look unusual. This gives the platform some coverage of new or modified attacks, which is a known gap in signature-based and purely supervised approaches [8].

The third is to **make results usable by a person**. Every module produces structured output instead of a bare yes/no answer: class labels with confidence values in the classification modules, and alerts with a severity level and the most unusual features in the anomaly module. This is the basis on which the planned dashboard and investigation agent will build.

Within the wider Final Year Project, the product is meant to show how separate AI components can contribute to a single defensive workflow. It is a research prototype tested on benchmark data in a controlled setting, not a replacement for an operational security operations centre (SOC).

1.2 Product Scope

The system is a software platform for analysing security data with machine learning. Its scope is defined by the twelve modules listed in the approved project proposal. Because this is a mid-term report, the scope is described together with the current state of each module, using four status labels: **Completed** (implemented, trained and evaluated as a standalone prototype), **Partially implemented**, **In progress** (development has started) and **Planned** (scheduled for the second half of the project).

Core functionality. The platform analyses three kinds of evidence. Network flow

records, described by statistics such as packet counts, packet lengths, durations and TCP flag counts, are classified as benign or as one of several attack families. They are also checked for deviations from normal traffic. Feature vectors extracted from Windows Portable Executable (PE) files are classified as benign or malicious, and malicious samples are assigned a malware category. System and application logs will be analysed by the log analysis module once it is completed.

Technological components. The implemented modules are written in Python. They use pandas and NumPy for data handling, scikit-learn [10] for preprocessing, Isolation Forest, Local Outlier Factor, Random Forest and the autoencoder, LightGBM [11] for network threat classification, CatBoost [12] for malware detection, joblib for model persistence and Gradio [13] for the malware module's interface. The development environment named in the proposal also includes FastAPI for backend services, React for the web dashboard and a relational database. These are planned for the integration phase and are not yet in use.

Limitations and boundaries. The system will not attack or scan real systems without permission, develop malware, control production systems automatically, or include hardware such as physical intrusion prevention devices. All development and testing take place in a controlled environment using public datasets: CIC-IDS2017 [14], NSL-KDD [15] and BODMAS [16]. The network module can generate recommended response plans (for example, rate limiting or blocking), but these are text outputs for analyst review and are never executed by the system. The current modules work on pre-extracted features: they do not capture live packets or parse raw executable files.

Integration and compatibility. At present the three completed modules are independent prototypes, each with its own interface (a command-line tool for network threat classification and for anomaly detection, and a Gradio web interface for malware classification). Each module already produces structured output, which will make integration easier. The anomaly module, for example, writes alerts in JSON format with source and destination addresses, a score, a severity level and the main contributing features. A shared backend that receives these outputs and passes them to the dashboard and the investigation agent is planned but not yet implemented.

Evolution and future enhancements. After integration, the platform can be extended with live data sources (a flow exporter such as CICFlowMeter, or log collectors), additional datasets such as UNSW-NB15 [17], better calibrated alert thresholds and the remaining analysis modules described below.

Modules

Module 1: Network Threat Classification (Completed, standalone prototype)

- Preparation of CIC-IDS2017 flow records and mapping of attack labels into six attack families
- Preprocessing fitted on training data only (median imputation, variance filtering, robust scaling)
- Stage 1: class-weighted LightGBM classifier for benign versus attack
- Stage 2: class-weighted LightGBM classifier for the attack family
- Supporting outputs: anomaly score, feature-based explanation, MITRE ATT&CK

context and a recommended (non-executed) response plan

Module 2: Malware Classification (Completed, standalone prototype)

- Preparation of the BODMAS feature data (2,326 numerical features per sample)
- Stage 1: CatBoost classifier for benign versus malware
- Stage 2: Random Forest classifier for eight malware categories
- Gradio interface for CSV-based batch prediction and result visualisation

Module 3: Anomaly Detection (Completed, standalone prototype)

- Preprocessing of flow features (cleaning, log transform, categorical encoding, robust scaling, optional removal of redundant features)
- Unsupervised detectors trained on benign traffic only: Isolation Forest, Local Outlier Factor, autoencoder and robust z-score
- Score combination and an alert threshold calibrated on held-out normal traffic
- Alert generation with severity levels and the most anomalous features as an explanation
- Evaluation tooling and a hybrid variant combining the anomaly detector with a supervised classifier

Module 4: Log Analysis (In progress)

- Examination of system and application logs for errors and suspicious events (design and implementation details will be reported in the final report)

Modules 5–12 (Planned)

- Network Monitoring: collection of live network activity for the analysis modules
- Incident Correlation: grouping related alerts and events into incidents
- Vulnerability Analysis: identification of known weaknesses in systems and applications
- Attack-Path Analysis: study of how an attacker could move through a system, with MITRE ATT&CK [18] as a reference
- Attack Prediction: estimation of likely future attack activity from available evidence
- Risk Analysis: prioritisation of threats and vulnerabilities by estimated risk
- Investigation Agent: an AI agent that combines module results and explains incidents in plain language
- Dashboard and Feedback: a web interface for results, plus analyst feedback on their quality

1.3 Objectives

The primary goal of the project is to build and evaluate an AI-based platform that helps security analysts detect, classify and understand threats across different kinds of

security data. This goal is divided into the following objectives. Objectives 1–5 belong to the first half of the project and are the subject of this mid-term report; objectives 6–9 belong to the second half.

O1. Network threat classification. Train a classifier that separates benign and malicious network flows and assigns attacks to families, and report class-level precision, recall and F1-score in addition to overall accuracy. *(Achieved for CIC-IDS2017.)*

O2. Malware classification. Train a two-stage model that detects malicious PE samples and assigns them to malware categories, using the BODMAS dataset. *(Achieved; metrics to be re-confirmed against the saved model files before the final report.)*

O3. Anomaly detection for unseen attacks. Build an unsupervised detector trained only on normal traffic, and measure how many attack types absent from its training data it can still detect. *(Achieved on NSL-KDD and CIC-IDS2017.)*

O4. Sound evaluation. Evaluate every model on held-out data that is never used for training or tuning, fit preprocessing on training data only, and report metrics suited to imbalanced data, such as macro F1-score and PR-AUC [19]. *(Achieved for the three modules.)*

O5. Explainable and structured output. Make each module return structured results (labels, confidence values, severity levels and the main contributing features) that later components can consume. *(Achieved at module level.)*

O6. Integration. Connect the modules through a shared backend so that their results can be stored, combined and displayed together. *(Planned.)*

O7. Extended analysis. Complete log analysis and develop incident correlation, vulnerability analysis, attack-path analysis, attack prediction and risk analysis. *(Log analysis in progress; others planned.)*

O8. Investigation support. Develop an investigation agent that summarises findings from several modules in plain language, linked to the underlying evidence. *(Planned.)*

O9. Usability and feedback. Provide a dashboard that presents alerts, incidents and risk levels, and lets analysts give feedback on the results. *(Planned.)*

In terms of quality, the project aims to keep false alarms at a level an analyst can manage and to make every reported number reproducible from saved code, configuration and data splits. In the long term, the platform is intended as a foundation that can be extended with new models and data sources without redesigning the existing modules.

1.4 Intended Market of Product

The product is aimed at organisations that need to monitor their own networks and endpoints but cannot afford, or do not need, a full commercial SIEM or XDR deployment.

Target audience. The main target audience is small and medium-sized organisations, such as software companies, educational institutions and offices of public bodies.

These organisations often have a small IT or security team responsible for many systems at once. A second audience is academic and training settings, where the platform's open, modular design makes it useful for teaching and for research on machine learning in cybersecurity.

Market reach. The prototype is developed with public, internationally used benchmark datasets and does not rely on any organisation's private data, so it is not tied to a particular country or sector. Its initial setting is local: universities and organisations in Pakistan that want an affordable tool for studying and improving their security monitoring.

Scalability. Because the modules are independent, an organisation could adopt only the parts it needs, for example only network analysis, and add others later. Retraining a module on an organisation's own data, adding new models, or connecting new data sources through the planned backend should not require changes to the other modules. Operational deployment would first require validation on the organisation's own traffic, as discussed later in this report.

1.5 Intended Users of the Product

The system is designed for people who examine security events as part of their work. Their technical background differs, so the planned interface will present results at more than one level of detail.

Primary users: security analysts. SOC analysts and IT security staff are the main users. They will use the network and malware classifiers to obtain specific labels for suspicious traffic and files, and the anomaly detector to find unusual flows that no rule describes. For each alert they will see a severity level and the features that contributed most to the decision, which should shorten the time needed to decide whether an alert deserves further investigation. Planned features for this group include incident correlation, the investigation agent and the ability to mark results as correct or incorrect.

Secondary users: IT administrators and managers. Network and system administrators in smaller organisations often handle security alongside other duties. They will mainly use summary views, such as the number and severity of alerts and the most affected hosts. Managers and team leads will use the planned risk analysis and dashboard summaries to decide which problems need attention first.

Secondary users: students and researchers. Students and researchers in AI and cybersecurity can use the modules to study detection methods on public datasets, compare models and repeat experiments, since each module's training and evaluation are documented and reproducible.

Collaboration and user experience. The planned dashboard is intended to give analysts and administrators a shared view of the same alerts and incidents, so that findings can be passed between roles without re-collecting evidence. The current prototypes already reflect this goal in a simple form. The malware module offers a Gradio web page for uploading CSV files and viewing predictions, while the network and anomaly modules provide command-line tools that write structured result files.

Chapter 2

Background and Literature Review

A literature review places a project within existing knowledge: it shows what has already been built, which methods are established, and where gaps remain. For a security project this means looking at two kinds of prior work. The first is the products that organisations already use to monitor their systems. The second is the research on machine learning for intrusion detection, anomaly detection and malware analysis, on which the project's models are based [6], [7], [20].

This chapter first reviews ten existing systems, covering commercial platforms, open-source tools and a research system, and compares their capabilities with those of the proposed system. It then describes how the proposed system is expected to be used, states the problem and its limitations, presents the proposed solution with the design and current results of the three implemented modules, and explains the software process model used to develop the project.

2.1 Existing System Description

Studying existing systems shows which capabilities are already well served, which design choices have worked in practice, and where gaps remain that a new system could address. This section first summarises the research on which the project draws. It then reviews ten existing systems: four commercial platforms (Splunk Enterprise Security, Microsoft Sentinel, CrowdStrike Falcon and Elastic Security), four open-source tools (Wazuh, Security Onion, Snort and Suricata), the Zeek network monitor and the research system Kitsune. They were chosen to cover the main approaches to detection: security information and event management (SIEM), endpoint detection, rule-based network intrusion detection and machine learning-based anomaly detection. The capability statements below are based on each system's official documentation or the published paper.

2.1.1 Related Academic Work

Machine learning for intrusion detection has been studied extensively. Buczak and Guven survey supervised and unsupervised methods for cyber intrusion detection and note how strongly their reported performance depends on the dataset used [6]. Anomaly detection, which models normal behaviour and flags deviations from it, is reviewed in general by Chandola *et al.* [7] and for networks by Ahmed *et al.* [21]. Widely used unsupervised algorithms include Isolation Forest [22], Local Outlier Factor [23] and autoencoders [24]. Kitsune applies an ensemble of autoencoders to network traffic in an online setting [25]. For classifying known attacks, tree ensembles such as Random Forest [26] and gradient-boosted trees, including LightGBM [11] and CatBoost [12], are strong baselines on tabular flow features. In malware analysis, Ucci *et al.* survey the use of machine learning on static and dynamic features [20], and the EMBER and BODMAS datasets provide standard feature representations of Windows PE files [16], [27].

Several studies question how such results are obtained. Sommer and Paxson argue that network anomaly detection is harder than other machine learning tasks, because attacks are rare, errors are costly and normal traffic varies widely [8]. Arp *et al.* identify recurring evaluation pitfalls in security research, such as data snooping and unrealistic assumptions about the data [9]. Engelen *et al.* report labelling and flow-construction problems in CIC-IDS2017 itself [28]. These studies shaped the evaluation protocol of this project: held-out test data, imbalance-aware metrics and the removal of duplicate records.

2.1.2 Review of Existing Systems

Wazuh [5]: Wazuh is an open-source security platform that combines log collection and analysis, intrusion detection, file integrity monitoring and vulnerability detection using agents installed on monitored endpoints.

The main features of the application are listed below:

- Rule-based collection and analysis of log data from many sources
- Malware detection using rootkit checks, YARA rules, VirusTotal and hash lists
- Mapping of alerts to MITRE ATT&CK tactics and techniques
- Active response scripts triggered by alerts on monitored endpoints

The limitations of the application are listed below:

- Detection is mainly rule-based; machine learning-based anomaly detection is not documented as a core capability
- No machine learning classification of network flows into attack families
- Network visibility depends on logs from other devices instead of its own traffic analysis

Splunk Enterprise Security [2]: Splunk Enterprise Security is a commercial SIEM built on the Splunk data platform, used to search, correlate and investigate security events across an organisation.

The main features of the application are listed below:

- Centralised collection, search and correlation of security data
- User and entity behaviour analytics that learn normal behaviour
- Coverage mapping against MITRE ATT&CK
- Built-in security orchestration, automation and response (SOAR)

The limitations of the application are listed below:

- Commercial licensing, which can be costly for small organisations
- Detection logic and models are not open for inspection or retraining by the user
- Network traffic is analysed through ingested logs, not captured directly

Microsoft Sentinel [3]: Microsoft Sentinel is a cloud-native SIEM and SOAR service on Microsoft Azure that collects data from cloud and on-premises sources and applies analytic rules and machine learning to detect threats.

The main features of the application are listed below:

- Analytic rules and anomaly detection over collected data
- Incident grouping and investigation tools
- MITRE ATT&CK coverage views
- Automation rules and playbooks for response

The limitations of the application are listed below:

- Tied to the Azure cloud and to usage-based commercial pricing
- Built-in machine learning models cannot be inspected
- Requires data connectors and configuration effort to reach full value

CrowdStrike Falcon [4]: CrowdStrike Falcon is a commercial, cloud-delivered platform centred on endpoint detection and response (EDR), extended with SIEM and SOAR capabilities.

The main features of the application are listed below:

- AI- and behaviour-based detection of malicious activity on endpoints
- Malware prevention and detection
- Mapping of detections to MITRE ATT&CK
- Remote real-time response and workflow automation

The limitations of the application are listed below:

- Proprietary and subscription-based
- Focused on endpoints; network flows are not analysed directly
- Detection models are not open for inspection or adaptation

Elastic Security [29]: Elastic Security is built on the Elasticsearch stack and combines SIEM, detection rules, machine learning anomaly detection jobs and endpoint protection.

The main features of the application are listed below:

- Detection rules and machine learning anomaly detection jobs
- Endpoint malware protection (Elastic Defend)
- MITRE ATT&CK coverage overview for its detection rules
- Host isolation and other response actions

The limitations of the application are listed below:

- Some capabilities, including response actions, require a paid subscription
- Deploying and tuning the stack requires considerable expertise
- No documented classification of network flows into attack families

Security Onion [30]: Security Onion is a free and open platform for network security monitoring, intrusion detection and log management that bundles tools such as Suricata, Zeek and Elasticsearch.

The main features of the application are listed below:

- Network intrusion detection alerts and full packet capture
- Network metadata from Zeek and Suricata
- File analysis with YARA rules
- Centralised log storage and case management

The limitations of the application are listed below:

- Detection relies mainly on the rules of its bundled tools
- Interpreting the large volume of collected data still depends on the analyst
- Machine learning-based classification is not a documented core feature

Snort [31]: Snort is a long-established open-source network intrusion detection and prevention system that inspects packets against a set of rules.

The main features of the application are listed below:

- Real-time packet inspection with rule-based detection
- Inline deployment to block malicious packets
- Free community ruleset and open-source engine

The limitations of the application are listed below:

- Detects only what its rules describe, so new attacks need new rules
- No machine learning-based anomaly detection
- No log analysis or incident management functions

Suricata [32]: Suricata is an open-source network intrusion detection, intrusion prevention and network security monitoring engine developed by the Open Information Security Foundation.

The main features of the application are listed below:

- Rule-based detection of network attacks
- Inline prevention mode
- Network security monitoring output, including protocol logs

The limitations of the application are listed below:

- Rule-based, with limited ability to detect unseen attacks
- No built-in machine learning classification
- Alerts must be interpreted and correlated in other tools

Zeek [33], [34]: Zeek (formerly Bro) is an open-source network analysis framework that turns traffic into detailed transaction logs and supports behaviour-based detection through its own scripting language.

The main features of the application are listed below:

- High-fidelity logs of network transactions across many protocols
- Scriptable, behaviour-based detection
- Widely used as a data source for other security tools

The limitations of the application are listed below:

- Detection logic must be written as scripts by experts
- No machine learning classification or malware analysis
- No automated response

Kitsune [25]: Kitsune is a research network intrusion detection system that uses an ensemble of autoencoders to detect anomalies in network traffic online and without supervision.

The main features of the application are listed below:

- Unsupervised learning of normal traffic, so no labelled attacks are needed
- Online operation on live packet streams
- Open-source reference implementation

The limitations of the application are listed below:

- Produces an anomaly score only, without naming the type of attack
- Addresses network anomalies only (no malware or log analysis)
- A research prototype without a user interface or response functions

Proposed System: The proposed system is a modular platform that applies machine learning to several kinds of security evidence. It combines supervised classification of

known threats with unsupervised detection of unseen behaviour, and returns explainable, structured results.

The main features of the application are listed below:

- Classification of network flows into attack families (implemented)
- Malware detection and malware category classification (implemented)
- Unsupervised anomaly detection with per-alert explanations (implemented)
- MITRE ATT&CK context for network threats (implemented)
- Log analysis (in progress); correlation, risk analysis, investigation agent and dashboard (planned)

Table 2.1 compares the reviewed systems with the proposed system. The proposed system is not intended to replace these mature platforms. Its contribution is to bring supervised classification of flows and malware together with unsupervised anomaly detection in one open, explainable design. None of the reviewed systems documents machine learning classification of network flows into attack families. The proposed system does not include rule-based detection, which remains a strength of the established tools, and it deliberately produces response recommendations instead of executing automatic responses.

Table 2.1: Applications Comparison (✓: documented capability; ○: partial, limited or unclear; ✗: not documented. For the proposed system, ✓ means implemented at midterm and ○ means in progress, planned or limited by design.)

Features	**Applications**										
	Wazuh [5]	Splunk ES [2]	MS Sentinel [3]	CrowdStrike [4]	Elastic Sec. [29]	Sec. Onion [30]	Snort [31]	Suricata [32]	Zeek [34]	Kitsune [25]	Proposed
Network traffic monitoring	○	○	○	○	○	✓	✓	✓	✓	✓	○
Signature / rule-based detection	✓	✓	✓	✓	✓	✓	✓	✓	○	✗	✗
ML / behavioural anomaly detection	○	✓	✓	✓	✓	○	✗	✗	✓	✓	✓
ML classification of flow attacks	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓
Malware detection / classification	✓	○	○	✓	✓	✓	○	○	○	✗	✓
Log analysis (SIEM)	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗	○
MITRE ATT&CK mapping	✓	✓	✓	✓	✓	○	✗	✗	✗	✗	✓
Open source or free	✓	✗	✗	✗	○	✓	✓	✓	✓	✓	○
Automated response	✓	✓	✓	✓	✓	○	✓	✓	✗	✗	○

2.2 Future System Usage Analysis

Once integrated, the system is expected to fit into the monitoring routine of a small security or IT team as an analysis layer that sits between raw data sources and the analyst. It is not intended to replace existing firewalls or log collectors.

Integration into the target environment. In an organisation, flow records would be produced by a flow exporter at the network edge, executable files or their extracted

features would come from endpoints or mail gateways, and logs would come from servers and applications. The platform would analyse each stream with the relevant module and present the results in one place. Because the modules work on standard representations (CICFlowMeter-style flow features and EMBER-style PE features), an organisation could connect them to its existing data collection rather than install new sensors. This requires feature extraction to match the formats the models were trained on, which is part of the planned integration work.

Impact on analysts. An analyst who today receives a generic alert would instead see a specific label, such as "PortScan" or "Ransomware", with a confidence value. Unusual flows that match no known category would still be reported, with the features that made them stand out. Once incident correlation is implemented, alerts belonging to the same activity would be grouped, and the investigation agent would summarise them in plain language. Together these changes should reduce the time needed to decide which events deserve attention.

Impact on administrators and managers. Administrators would receive summary views of alert volume and severity, and managers would receive risk-based priorities. This supports the kind of prioritisation recommended in incident response guidance, where limited staff must decide which incidents to handle first [1].

Use of advanced technologies. The platform combines gradient-boosted trees, Random Forests and unsupervised anomaly detection, and is planned to use an AI agent for explanation. The value of this combination is that each technique covers a different weakness: supervised models are precise on known threats, anomaly detection covers unknown behaviour, and the agent and dashboard make the combined output understandable.

Long-term evolution. The modular design allows models to be retrained on new data as attacks change, and allows new modules to be added. Evaluation results already show that thresholds tuned on one network do not transfer perfectly to another (see Section 2.4.5 and the anomaly results in Section 2.4). Future versions will therefore need periodic recalibration on the organisation's own traffic, and the analyst feedback feature is planned partly for this purpose.

Wider influence. In academic settings, the platform can serve as a reproducible reference for comparing detection methods on public datasets. In organisations, it can encourage a habit of evidence-based triage, where alerts are accompanied by the reasons behind them.

2.3 Problem Statement / Limitations

Core problem. Security teams must examine large and varied streams of evidence (network traffic, files and logs) to find the small number of events that represent real attacks. Existing tools collect and store this evidence well, but much of the interpretation remains manual. Rule- and signature-based detection only recognises patterns that have already been written down. Supervised machine learning only recognises the classes present in its training data. Neither reliably flags new behaviour [8]. Different kinds of evidence are also usually analysed by different tools, so the analyst has to connect them by hand.

Impact on stakeholders. For analysts, this means time spent on alerts that turn out to be harmless, and a risk of missing activity that matches no existing rule. For administrators in smaller organisations, who handle security alongside other duties, the effort needed to operate several separate tools is often too high. For the organisation, slow or missed detection increases the potential damage of an incident.

Deficiencies of current solutions. The review in Section 2.1 shows the following:

- Commercial platforms offer broad coverage but are costly and closed, so their detection logic cannot be inspected or adapted.
- Open-source tools are accessible but mostly rule-based.
- Research systems demonstrate strong algorithms but usually address a single task, for example only network anomaly detection or only malware classification, and are rarely evaluated together.
- Results reported on benchmark datasets frequently overstate real-world performance because of dataset artefacts and evaluation pitfalls, such as duplicate records shared between training and test data [9], [28].

Problem statement. There is a need for a modular, transparent system that applies machine learning to several kinds of security evidence, covers both known and previously unseen threats, explains its findings, and is evaluated honestly enough that its results can be trusted.

Technical challenges and limitations.

- *Dataset realism.* The modules are trained on public benchmarks recorded in laboratory conditions (CIC-IDS2017, NSL-KDD and BODMAS). Results on these datasets may not transfer to a different network or to future malware [8], [28].
- *Class imbalance and rare classes.* Some attack families have very few examples, for example 36 Infiltration flows in CIC-IDS2017 and 192 Virus samples in BODMAS. Results on these classes are statistically weak.
- *False alarms.* Anomaly detection trades false alarms against missed attacks. The improved detector's false-positive rate rose to 11.9% on NSL-KDD test traffic, which would be too high for many operational settings without recalibration.
- *Integration.* The modules were developed independently with different interfaces and data formats, and must be connected through a common backend and data schema.
- *Security and robustness.* An integrated system that stores security data must itself be secured. Machine learning models can also be targeted by adversarial manipulation, which is outside the current scope.

Planned mitigation. The project addresses these challenges by:

- evaluating every model on held-out data never used for training or tuning;
- reporting class-level and imbalance-aware metrics instead of accuracy alone;
- removing duplicates where they cause leakage;

- keeping thresholds configurable for recalibration;
- producing response recommendations instead of automatic actions.

The incremental development approach described in Section 2.5 allows these issues to be addressed step by step, with each increment tested before the next begins.

2.4 Proposed Solution

The proposed solution is a modular, AI-based platform in which each module analyses one kind of security evidence and returns a structured finding. The platform is designed around two complementary ideas. *Supervised* models give precise labels for threats they have learned, while an *unsupervised* anomaly detector covers behaviour that no model has seen before. Their outputs are then meant to be combined, correlated and explained to the analyst. This section describes the overall architecture, the three implemented modules with their current results, the remaining planned components, and the main design considerations.

2.4.1 System Architecture and Workflow

Figure 2.1 shows the intended architecture. Solid boxes are components that have been implemented and evaluated, the dashed orange box is the log analysis module now in development, and dashed grey boxes are planned components. The intended workflow is as follows:

1. Security evidence enters the platform: network flow records, PE malware feature vectors and logs.
2. Each analysis module processes the evidence it is responsible for and produces a structured result, such as a class label with confidence, or an alert with a severity level and explanation.
3. The planned backend stores these results and passes them to the correlation, risk and investigation components.
4. The dashboard presents incidents, risk levels and explanations to the analyst, whose feedback is recorded to evaluate and improve the system.

At the mid-term stage, steps 1 and 2 are implemented for the three completed modules, which run as standalone prototypes on benchmark data. Steps 3 and 4 are planned.

The three completed modules do not yet exchange data with each other. They were developed independently, each by one team member, and they share a design principle: every module produces machine-readable output that the planned backend can consume. The network and anomaly modules both analyse flow records but answer different questions. The network module asks *which known attack does this flow resemble?*, while the anomaly module asks *how unusual is this flow compared with normal traffic?* Section 2.4.5 discusses how the two are expected to complement each other.

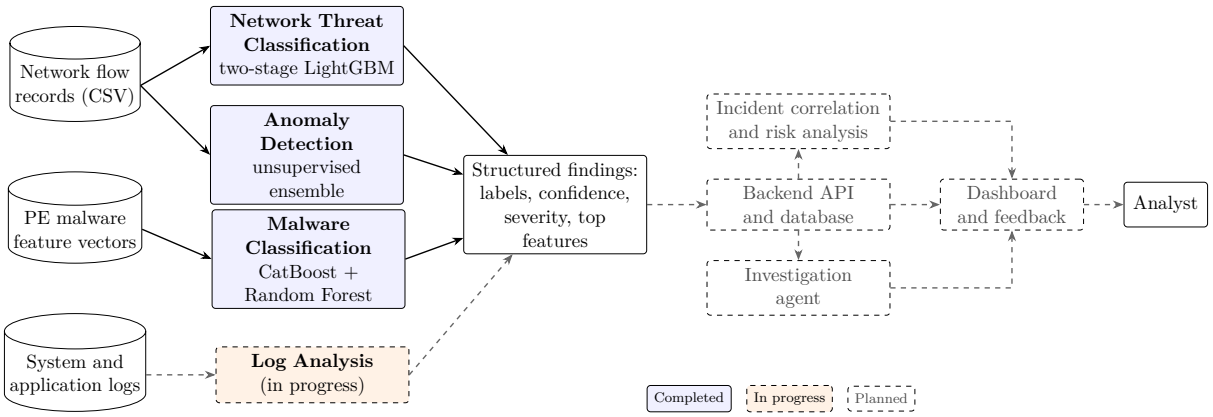


Figure 2.1: Overall architecture of the AI-Based Autonomous Cyber Defense System and implementation status at mid-term

2.4.2 Module 1: Network Threat Classification

Role. This module labels network flows as benign or as one of six attack families, so that the analyst sees what kind of activity is taking place, not just that something is wrong.

Dataset. The module uses the CIC-IDS2017 dataset published by the Canadian Institute for Cybersecurity [14], [35]. It contains five days of benign and attack traffic, recorded as flows by the CICFlowMeter tool. All eight flow files were used, giving 2,830,743 usable flows described by 77 numeric features. Source and destination addresses, ports and timestamps were excluded so that the model cannot rely on identifiers. The original labels were grouped into seven classes (Table 2.2). In the module's taxonomy, the *WebAttack* group also contains the FTP-Patator and SSH-Patator brute-force attacks and the Heartbleed exploit. The data is highly imbalanced: about 80.3% of flows are benign, and only 36 Infiltration flows exist in the entire dataset.

Table 2.2: Class distribution of the prepared CIC-IDS2017 data used by the Network Threat Classification module

Class	All rows	Train rows	Test rows
BENIGN	2,273,097	1,826,507	446,590
DoS	252,661	206,951	45,710
PortScan	158,930	129,834	29,096
DDoS	128,027	102,360	25,667
WebAttack	16,026	13,048	2,978
Botnet	1,966	1,582	384
Infiltration	36	29	7
Total	2,830,743	2,280,311	550,432

Method. Preprocessing converts all features to numbers, replaces infinite values, imputes missing values with the training median, removes low-variance features and applies robust scaling. Each step is fitted on training data only, to avoid information leakage. The split keeps identical feature vectors in the same partition, using a fingerprint of se-

lected flow fields. The classifier is a two-stage cascade of LightGBM gradient-boosted tree models [11] (Figure 2.2). Stage 1 decides between benign and attack, using a decision threshold of 0.7723 chosen on a validation subset. Flows predicted as attacks pass to Stage 2, which assigns one of the six families. Both models use balanced class weights instead of oversampling. Their settings are a learning rate of 0.05, 31 leaves, a minimum of 30 samples per leaf, 85% feature and row sampling, and L1 and L2 regularisation of 0.1 and 2.0. Early stopping on a validation subset selected 600 trees for Stage 1, which is the configured maximum, and 256 trees for Stage 2. The module also attaches an Isolation Forest anomaly score, a feature-importance explanation, MITRE ATT&CK context [18] and a recommended response plan to each decision. The response plan is produced as text only and is never executed.

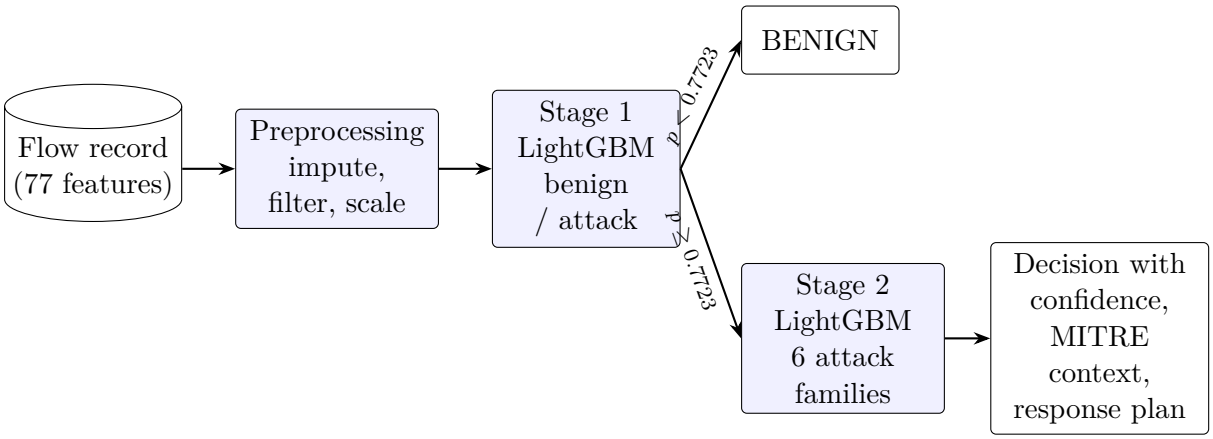


Figure 2.2: Two-stage workflow of the Network Threat Classification module

Results. The cascade was evaluated once on the 550,432-flow holdout set (Table 2.3). The binary gate reached 99.70% attack precision and 99.84% attack recall. Of the 446,590 benign test flows, 313 were wrongly flagged, and of the 103,842 attack flows, 164 were missed. The end-to-end cascade achieved 99.91% accuracy with a macro F1-score of 0.9578. The gap between this value and the weighted F1-score of 0.9991 reflects weaker results on rare classes. Botnet recall is 71.35% (274 of 384 test flows), and the Infiltration result of six correct out of seven is too small to be statistically reliable. Figure 2.3 shows the confusion matrices.

Table 2.3: Holdout performance of the Network Threat Classification module (CIC-IDS2017)

Evaluation	**Rows**	**Accuracy**	**Macro F1**	**Weighted F1**
Stage 1 (benign vs. attack)	550,432	99.91%	0.9986	0.9991
Stage 2 on true attacks only	103,842	99.98%	0.9997	0.9998
End-to-end cascade	550,432	99.91%	0.9578	0.9991

Limitations. CIC-IDS2017 was recorded in a single laboratory network over five days, so these results do not show how the model would perform on a different network or on future traffic. Previous work has also reported labelling and flow-construction problems in this dataset [28]. The rare classes (Botnet and Infiltration) have too few examples for reliable estimates. Stage 1 used the maximum number of trees allowed, so a wider search might improve it. The attached Isolation Forest score has not been

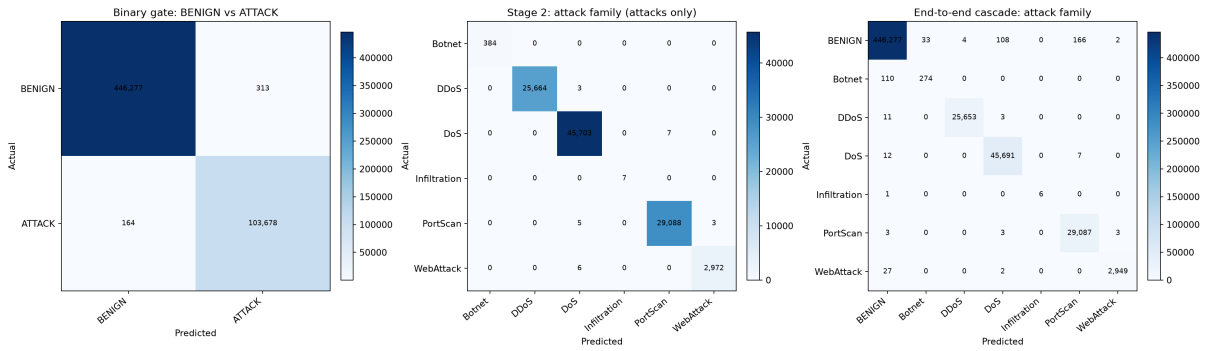


Figure 2.3: Confusion matrices of the Network Threat Classification module on the hold-out set: binary gate (left), Stage 2 on true attacks (centre) and end-to-end cascade (right)

validated against labelled data and is not presented as a detector of new attacks. The current interface is a command-line tool and Python API; live packet capture and response execution are outside the verified scope.

2.4.3 Module 2: Malware Classification

Role. This module decides whether a Windows executable is benign or malicious and, if it is malicious, which category of malware it belongs to. This tells the analyst both whether a file is dangerous and what kind of behaviour to expect from it.

Dataset. The module uses BODMAS, a dataset of Windows PE samples released for research on learning-based malware analysis [16]. The cleaned version used contains 134,428 samples (77,138 benign and 57,290 malicious), each described by 2,326 numerical static features. The feature format follows the EMBER feature set [27]. Category labels were joined to the malicious samples, and eight categories were selected for the second stage (Table 2.4). Very small categories were left out.

Table 2.4: Malware categories used by the second stage of the Malware Classification module (BODMAS)

Category	Samples	Category	Samples
Trojan	29,972	Ransomware	821
Worm	16,695	Dropper	715
Backdoor	7,331	Information stealer	447
Downloader	1,031	Virus	192

Method. Both stages use a stratified 80/20 train–test split with a fixed random seed of 42 (Figure 2.4). No feature scaling is applied, which suits tree-based models, and no synthetic oversampling such as SMOTE was used. Stage 1 is a CatBoost gradient-boosting classifier [12] with 300 iterations, depth 6 and a learning rate of 0.1. Stage 2 is a Random Forest [26] with 100 trees, square-root feature sampling, at least two samples per leaf, and `balanced_subsample` class weighting to reduce the effect of category imbalance. A Gradio interface [13] loads the saved models, accepts a CSV file containing the 2,326 features, and displays predictions, confusion matrices and summary charts.

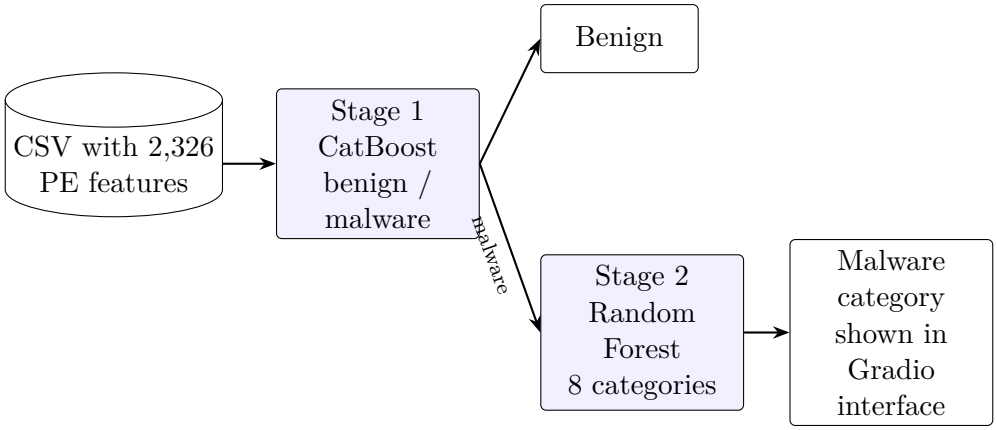


Figure 2.4: Two-stage workflow of the Malware Classification module

Results. Table 2.5 lists the results reported in the module's documentation. On the 26,886-sample test set, the CatBoost detector misclassified 62 benign samples as malware and missed 71 malicious samples. The Random Forest category classifier reached 94.34% accuracy and a macro F1-score of 0.8766. The lower macro value reflects weaker results on small categories, especially Virus, which has only 192 samples in the whole dataset.

Table 2.5: Test results of the Malware Classification module (as reported in the module documentation)

Stage	**Metric**	**Value**
Stage 1: CatBoost (benign vs. malware)	Accuracy	99.51%
	Precision (malware)	99.46%
	Recall (malware)	99.38%
	F1-score (malware)	99.42%
	ROC-AUC	99.97%
Stage 2: Random Forest (8 categories)	Accuracy	94.34%
	Macro F1-score	87.66%
	Weighted F1-score	94.32%

Limitations. The module classifies pre-extracted feature vectors. It does not yet extract features from raw executable files, scan endpoints in real time, or quarantine files. The Stage 2 result was measured on samples already known to be malicious, so it shows how well categories are separated, not the error of the full two-stage pipeline. The two stages were also evaluated on separate splits. Metrics from earlier experiments, such as a CatBoost run with 99.00% accuracy, are not part of the final pipeline. The values in Table 2.5 must be re-confirmed against the saved metrics file before the final report. Finally, malware evolves over time, and a random split may overestimate performance on future samples. Temporal evaluation, which BODMAS was designed to support [16], is left for future work.

2.4.4 Module 3: Anomaly Detection

Role. This module detects network activity that deviates from normal behaviour, whether or not it matches a known attack. It is trained only on benign traffic, so it needs no attack labels to learn and can flag attack types that the supervised classifiers have never seen [7], [21]. Each alert includes a severity level and the features that were most abnormal, so that the analyst can see why it was raised.

Datasets and protocol. The module was evaluated on NSL-KDD [15] and CIC-IDS2017 [14]. NSL-KDD uses its official training and test files. Its test file contains 17 attack types (3,750 of 12,833 attack records) that never appear in training, which makes it a direct test of detecting unseen attacks. For CIC-IDS2017, an analysis found that 10.9% of all rows (23.6% of attack rows) are exact duplicates, and 1,396 rows have identical features but conflicting labels. Duplicate flows split across training and test sets would inflate results. The module therefore removes duplicates and conflicting rows before splitting, then draws a fixed 10% sample and splits it 70/30 with stratification (176,467 training and 75,630 test flows). All design choices were made on a validation split carved from the training data. The test sets were used only for the final measurements, which are averaged over three random seeds.

Method. The pipeline is shown in Figure 2.5:

- **Preprocessing**, fitted on benign training data: median imputation, a signed logarithmic transform for heavy-tailed counts, one-hot encoding, robust scaling, and, for CIC-IDS2017, removal of features with absolute correlation above 0.99 with an already kept feature (70 reduced to 48 features).
- **Detectors.** Four unsupervised detectors are available: an Isolation Forest [22], the Local Outlier Factor (LOF) [23], an autoencoder that scores flows by reconstruction error [24], and a robust z-score baseline.
- **Score combination.** Each detector's score is converted to its rank among the scores of held-out benign traffic, and the selected detectors are combined.
- **Threshold.** An alert is raised when the combined score exceeds a percentile of the benign scores.

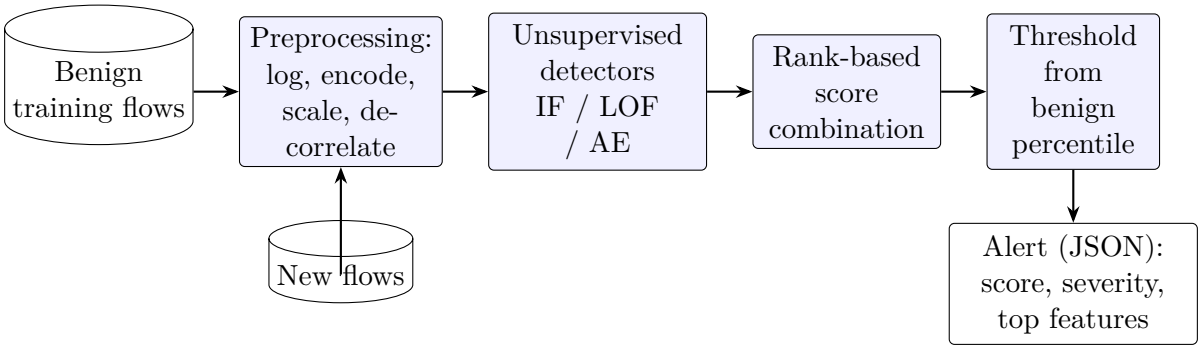


Figure 2.5: Workflow of the Anomaly Detection module

The first version (v1) averaged all four detectors and used a fixed 99th-percentile threshold. Its evaluation exposed clear weaknesses. On CIC-IDS2017 it detected only 3.9% of attacks, because weak detectors diluted the combined score and the threshold

was too strict. An improved version (v2) was then tuned in stages, entirely on validation data:

1. removing redundant features;
2. a grid search over each detector's settings;
3. a search over all 60 combinations of detector subset, normalisation and combination rule;
4. selection of the alert percentile by F1-score.

A change was adopted only if it improved the validation result. Table 2.6 shows the selected configurations. A *hybrid* variant was also evaluated. It adds a supervised Random Forest [26] for known attacks and flags a flow if either part flags it.

Table 2.6: Configuration of the improved (v2) anomaly detector selected on validation data

Setting	**NSL-KDD**	**CIC-IDS2017**
Detectors combined	Isolation Forest + LOF	LOF + autoencoder
Combination rule	maximum of ranks	mean of ranks
LOF	10 neighbours, 30,000 reference flows	10 neighbours, 30,000 reference flows
Isolation Forest	4,096 samples per tree, 50% of features	(not used)
Autoencoder	(not used)	up to 200 epochs, early stopping
Alert threshold	95th percentile	97th percentile

Results. Table 2.7 compares the original detector, the improved detector and the hybrid on the untouched test sets. On CIC-IDS2017, the improved detector raised the F1-score from 0.071 to 0.835 and the PR-AUC from 0.534 to 0.891, while raising the false-positive rate from 0.9% to 2.9%. On NSL-KDD the F1-score rose from 0.522 to 0.899, but this gain came mainly from a lower threshold. The false-positive rate rose from 5.5% to 11.9%, and the threshold-independent PR-AUC was about 0.01 lower than v1. The most important result for the platform concerns unseen attacks. On the 17 NSL-KDD attack types absent from training, recall rose from 41.2% (v1) to 84.5% (v2) at seed 42. The hybrid's supervised part added almost nothing there (85.1%), as expected, since a supervised model cannot recognise new attack types. Figure 2.6 shows the CIC-IDS2017 comparison.

Table 2.7: Anomaly Detection test results: original (v1), improved (v2) and hybrid (mean of three seeds)

Dataset	Model	F1-score	Recall	False-positive rate	PR-AUC
NSL-KDD	Original (v1)	0.522	36.9%	5.5%	0.911
	Improved (v2)	0.899	89.1%	11.9%	0.901
	Hybrid	0.904	90.0%	12.1%	0.956
CIC-IDS2017	Original (v1)	0.071	3.9%	0.9%	0.534
	Improved (v2)	0.835	82.2%	2.9%	0.891
	Hybrid	0.931	99.7%	3.0%	0.998

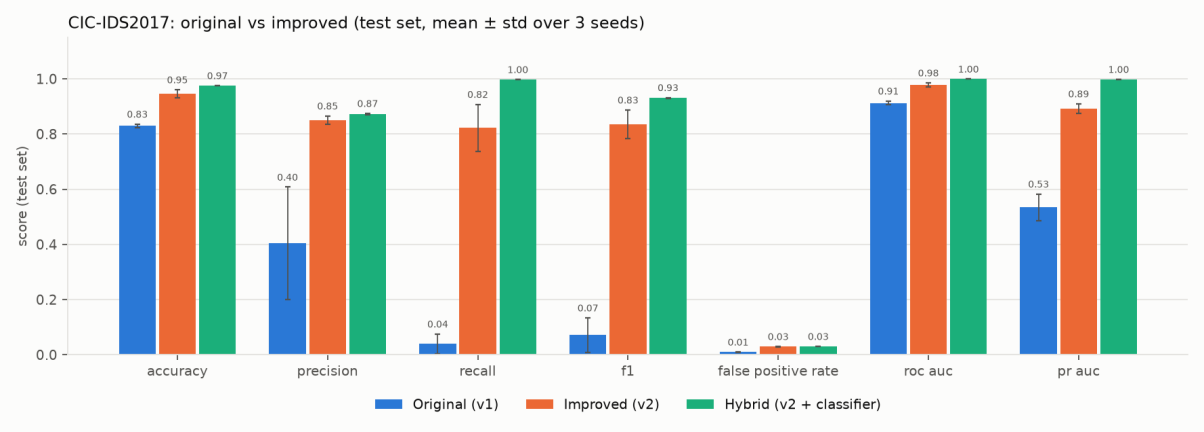


Figure 2.6: Anomaly Detection on CIC-IDS2017: original vs. improved vs. hybrid (test set, mean ± standard deviation over three seeds)

Limitations. The rise in false positives on NSL-KDD comes from a known difference between the normal traffic in its training and test files. The training false-positive rate was 4.6%, against 11.8% on test, so thresholds will need recalibration on each deployment network. On CIC-IDS2017, the v2 detector still rarely detects SSH-Patator, web attacks and Bot traffic, which look statistically similar to normal flows. The F1-score of v2 on CIC-IDS2017 varied between 0.776 and 0.876 across seeds. Tuning used a single validation split instead of cross-validation. Before this protocol was fixed, an early exploratory analysis had examined CIC-IDS2017 test results; this is disclosed as a possible source of bias.

2.4.5 Relationship Between the Modules

The three modules address different parts of the same defensive task. Network Threat Classification and Malware Classification are supervised. They are precise on the attack families and malware categories present in their training data, but they cannot recognise genuinely new behaviour. Anomaly Detection is unsupervised, so it is less precise but can flag behaviour that has never been labelled. The anomaly module's evaluation supports this division of labour: its supervised component lifted recall on known CIC-IDS2017 attacks from 82% to 99.7%, while on unseen NSL-KDD attacks the unsupervised detector provided almost all of the coverage. In the integrated system, the network classifier is expected to take the supervised role for flows and the anomaly detector the unsupervised role, and their outputs will be combined by the planned correlation component. This

combination has not yet been implemented or tested.

Two points must be kept in mind when comparing the modules. First, the network classifier also produces an Isolation Forest anomaly score. That score is an unvalidated auxiliary signal, whereas the Anomaly Detection module is the dedicated, evaluated component for this task. Second, both flow modules use CIC-IDS2017 but with different protocols. The network module keeps duplicate flows (grouped within one partition) and uses the full dataset, while the anomaly module removes duplicates and uses a 10% sample. Their numbers are therefore not directly comparable, and the integration phase will need a common evaluation split.

2.4.6 Planned Components

Table 2.8 summarises the implementation status of all proposed components. Log analysis is under development. The backend API and database (FastAPI and a relational database are planned), the dashboard (React is planned), incident correlation, risk analysis and the investigation agent will be developed in later increments. The agent will follow the main steps of incident handling described in NIST guidance [1].

Table 2.8: Implementation status of the proposed components at mid-term

Component	**Status**	**Current form**
Network Threat Classification	Completed	Prototype; CLI and Python API
Malware Classification	Completed	Prototype; Gradio interface
Anomaly Detection	Completed	Prototype; CLI and Python API
Log Analysis	In progress	Under development
Network Monitoring (live capture)	Planned	–
Incident Correlation	Planned	–
Vulnerability Analysis	Planned	–
Attack-Path Analysis	Planned	–
Attack Prediction	Planned	–
Risk Analysis	Planned	–
Investigation Agent	Planned	–
Dashboard and Feedback	Planned	–
Integration backend and database	Planned	–

2.4.7 Design Considerations

User interface and experience. The planned dashboard will group alerts into incidents and show, for each one, the module that raised it, its severity, its confidence and the features behind it. The aim is that an analyst can understand an alert without opening several tools. The Gradio interface of the malware module is an early example of this approach.

Technical implementation. All modules are implemented in Python with widely used libraries, and each saves its trained models and preprocessing objects so that it can be reloaded for inference without retraining. The planned backend will expose each

module through a common interface, so that adding or replacing a model does not affect the others.

Security and data privacy. The modules are trained only on public datasets and store no personal data. In a deployment, flow records and logs may contain sensitive information, so the planned backend will need access control, and stored results will need protection. The decision to produce response recommendations instead of executing them is also a safety measure, since an incorrect automatic block could disrupt legitimate services.

Customisation and flexibility. Thresholds, detector choices and model settings are kept in configuration files rather than in code. The anomaly module, for example, stores its tuned settings for each dataset in YAML files. An organisation can therefore adjust its false-alarm budget or retrain a module on its own data.

Anticipated impact. The completed modules show that the platform can give specific labels for known network attacks and malware, and can still flag a large share of previously unseen attacks. Once integrated, these results should reduce the time analysts spend on initial triage, provided that the models are validated on the target network first.

2.5 Software Process Model

A software process model is a structured description of the activities used to develop a system, such as requirements analysis, design, implementation, testing and deployment, and of the order in which they are carried out [36]. Common models include the Waterfall model, in which phases follow one another strictly; Agile methods, which work in short, flexible iterations; the Spiral model, which focuses on risk; and the Incremental model, in which the system is delivered as a series of increments, each adding working functionality [36], [37].

Chosen model. The project follows the **Incremental model**. The system is divided into parts that are specified, built and tested in separate increments, each producing working functionality that later increments extend. Within each increment, the usual activities of analysis, design, implementation and testing are carried out in sequence (Figure 2.7).

Justification.

- *Project complexity.* The proposed system has twelve modules. Building them all at once would be risky; developing them in increments allows each to be finished and evaluated before the next is started.
- *Team structure.* The first increment could be divided naturally among the three team members, each responsible for one analysis module. This is how the network, malware and anomaly modules were developed in parallel.
- *Flexibility.* Results from one increment inform the next. For example, the anomaly module's evaluation exposed duplicate records in CIC-IDS2017 and the need for recalibrated thresholds, and these findings will shape the integration work.
- *Timeline.* The academic calendar requires demonstrable progress at mid-term. The

Incremental model guarantees that a tested subset of the system exists at that point.

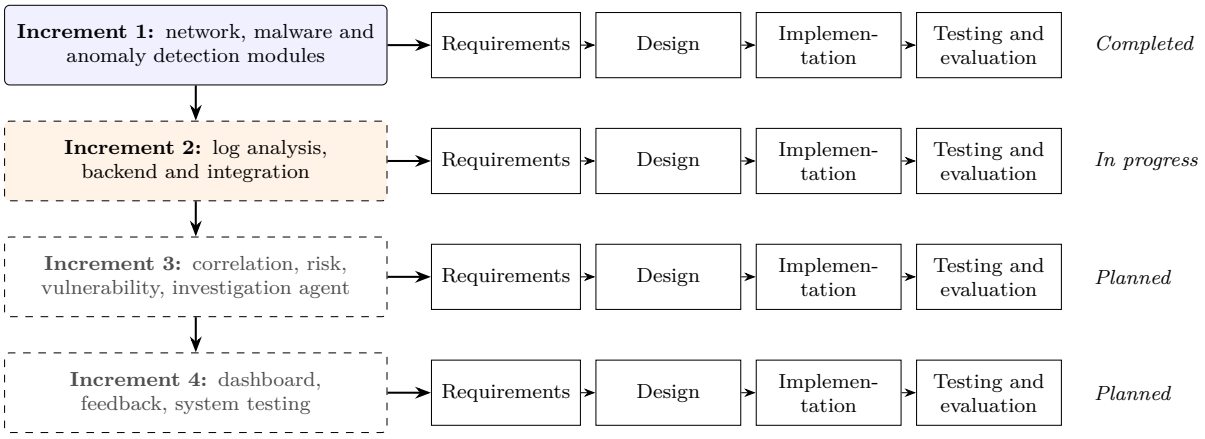


Figure 2.7: Incremental development model applied to the project, with the status of each increment at mid-term

Stages within each increment.

- *Requirements analysis.* The requirements for the increment are drawn from the proposal, the literature, dataset documentation and supervisor meetings (Section 3.4).
- *Design.* Each module's data pipeline, model architecture and output format are designed. Integration increments also design the shared interfaces.
- *Implementation.* Modules are implemented in Python and their trained models are saved for reuse.
- *Testing and evaluation.* Each model is evaluated on held-out data with metrics suited to imbalanced classes. Code is checked with unit tests where available; the anomaly module, for example, has an automated test suite. Integration and system testing will be carried out in Increments 2–4.
- *Deployment and maintenance.* Deployment is limited to a controlled demonstration environment. Maintenance consists of retraining models and recalibrating thresholds as new data becomes available.

Advantages for this project. Each increment ends with something that works and can be measured, so problems are found early. In Increment 1, for example, evaluation revealed weak results in the first version of the anomaly detector and duplicate data in CIC-IDS2017, and both were addressed before mid-term. Regular supervisor reviews at the end of each increment keep the work aligned with the project goals.

Challenges and mitigation. The main risk of incremental development is that independently built parts may be difficult to integrate later. Increment 1 has indeed produced three modules with different interfaces and data formats. This is mitigated by making integration the focus of Increment 2, before further modules are added, and by requiring every module to produce structured output.

Chapter 3

Software Requirements Specification

A Software Requirements Specification (SRS) records what a software system must do and the qualities it must have. Functional requirements describe the behaviour the system provides, and non-functional requirements describe properties such as performance, reliability, usability and security [36], [38]. A clear SRS gives all stakeholders a shared reference: developers know what to build, testers know what to verify, and supervisors can judge progress against agreed expectations. It also limits scope creep, because new ideas can be checked against what was actually agreed.

For a project developed in increments, the SRS also shows which requirements have already been met. This chapter introduces the document's scope and audience, lists the functional requirements of the implemented modules and the planned system with their current status, defines the non-functional requirements, describes the techniques used to gather the requirements, and presents the project time frame.

3.1 Introduction

This SRS describes the requirements of the AI-Based Autonomous Cyber Defense System. It guides the remaining development work and gives the team, the supervisor and the evaluation committee a common reference for what the system is expected to do. Because three modules are already implemented, the requirements in this chapter are of two kinds. Some describe behaviour that exists and has been tested on benchmark data. Others describe behaviour planned for later increments. Every requirement is labelled accordingly. The requirements were derived from the approved proposal and refined through the literature review, analysis of the datasets and their documentation, and regular supervisor meetings (Section 3.4).

3.1.1 Document Scope

The document covers both functional and non-functional requirements. The functional requirements state the inputs each module accepts, the processing it performs and the outputs it produces, together with the system-level functions planned for integration: storage, correlation, risk analysis, investigation support and the dashboard. The non-functional requirements set expectations for quality attributes, including accuracy, explainability, usability, maintainability and interoperability, as well as performance, safety, privacy and reproducibility. The SRS focuses on *what* the system must do and *why*. The internal design of the models is described in Chapter 2. Detailed design artefacts, such as use cases and sequence diagrams, belong to the next stage of the project.

3.1.2 Audience

The SRS is written for the following readers:

- **Development team:** the three team members, who use the SRS to keep their

modules consistent with each other and to plan the integration increment.

- **Supervisor and FYP committee:** who use it to assess the scope of the project and the progress made against agreed requirements.
- **Testing team:** in this project, the developers themselves, who derive unit, integration and system test cases from the listed requirements.
- **End users (security analysts and administrators):** whose needs, as identified in Chapter 1, are expressed in the functional and usability requirements.
- **Documentation team:** who use the SRS as the basis for user guides and the final report.
- **User experience designers:** who use the dashboard and feedback requirements to design the planned interface.
- **Future maintenance team:** who need to understand the expected behaviour of each module before retraining models or adding new ones.

3.2 Functional Requirements

Functional requirements define the services a system provides: the inputs it accepts, how it processes them and the outputs it returns [36]. They are the basis for design, for test cases and for checking at the end of the project whether the system does what was agreed. Each requirement below is written to be specific enough to be tested.

Table 3.1 lists the functional requirements, grouped by module. The *Status* column shows the state at mid-term: **Done** means implemented and tested on benchmark data, **Partial** means implemented with a known limitation, **WIP** means in progress, and **Planned** means scheduled for a later increment.

Table 3.1: Functional Requirements

Requirement ID	Description	Status
Network Threat Classification		
FR-N1	The module shall accept network flow records containing the CICFlowMeter-style features used in training, either as a single record or as a CSV batch.	Done
FR-N2	The module shall apply the saved preprocessing (imputation, variance filtering and scaling) fitted on the training data before classification.	Done
FR-N3	The module shall classify each flow as BENIGN or ATTACK using the Stage 1 model and the validation-selected decision threshold.	Done
FR-N4	For flows classified as ATTACK, the module shall assign one of six attack families (DDoS, DoS, PortScan, WebAttack, Botnet, Infiltration).	Done
FR-N5	The module shall return, for each flow, the predicted class, a confidence value, an anomaly score, a feature-based explanation and the related MITRE ATT&CK context.	Done
FR-N6	The module shall generate a recommended response plan for each decision as text for analyst review, and shall not execute it.	Done
FR-N7	The module shall provide command-line functions for training, evaluation and batch classification of CSV files.	Done

Continued on next page

Table 3.1 – *Continued from previous page*

Requirement ID	Description	Status
Malware Classification		
FR-M1	The module shall accept a CSV file containing the 2,326 static PE features expected by the trained models.	Done
FR-M2	The module shall reject input files that do not contain all required feature columns and report how many are missing.	Done
FR-M3	The module shall classify each sample as benign or malware using the CatBoost model.	Done
FR-M4	For samples classified as malware, the module shall predict one of eight categories using the Random Forest model.	Done
FR-M5	The module shall display predictions, model performance figures and summary charts in a web interface (Gradio).	Done
FR-M6	The module shall load saved models at start-up without retraining.	Done
FR-M7	The module shall extract features directly from raw executable files.	Planned
Anomaly Detection		
FR-A1	The module shall train on benign traffic only. When labels are present, attack rows shall be excluded from training automatically.	Done
FR-A2	The module shall preprocess flow features with steps fitted on training data only, including handling of missing and infinite values and categorical encoding.	Done
FR-A3	The module shall score each flow with the configured unsupervised detectors and combine their scores into a single anomaly score.	Done
FR-A4	The module shall flag a flow as anomalous when its score exceeds a threshold calibrated as a configurable percentile of held-out benign scores.	Done
FR-A5	The module shall write alerts in JSON or CSV format containing the time stamp, source and destination address, destination port, score, threshold, severity level (low, medium or high) and the three most anomalous features.	Done
FR-A6	The module shall evaluate a trained model on labelled data and report precision, recall, F1-score, false-positive rate, ROC-AUC, PR-AUC and recall per attack type, with ROC and precision–recall plots.	Done
FR-A7	The module shall read detector choices, thresholds and model settings from a configuration file, and shall save and reload trained models.	Done
FR-A8	The module shall validate the schema of input files and warn about missing or unexpected feature columns. (At present, missing columns are filled with training medians without a warning.)	Partial
Log Analysis		
FR-L1	The module shall read system and application logs and identify errors and suspicious events relevant to security incidents.	WIP
System Integration and Planned Components		
FR-S1	The system shall provide a backend service through which each analysis module can be called and its structured results returned in a common format.	Planned
FR-S2	The system shall store alerts, predictions and analyst feedback in a database.	Planned
FR-S3	The system shall correlate related alerts from different modules into incidents.	Planned

Continued on next page

Table 3.1 – *Continued from previous page*

Requirement ID	Description	Status
FR-S4	The system shall estimate a risk level for each incident to support prioritisation.	Planned
FR-S5	The system shall identify known vulnerabilities and possible attack paths, using MITRE ATT&CK techniques as a reference.	Planned
FR-S6	The investigation agent shall summarise an incident in plain language and link each statement to the module results it is based on.	Planned
FR-S7	The dashboard shall display alerts, incidents, risk levels and explanations, with filtering by time, module and severity.	Planned
FR-S8	The dashboard shall allow analysts to mark results as correct or incorrect and record this feedback for evaluation.	Planned
FR-S9	The system shall authenticate users and restrict access to stored security data according to their role.	Planned

3.3 Non-Functional Requirements

Non-functional requirements describe how well the system must perform its functions, not what those functions are [36], [38]. For a security tool they are especially important. A detector that finds every attack but raises thousands of false alarms, or that cannot explain its alerts, will not be used in practice. The requirements below set expectations for the proposed system. Where a module has already been measured, the current value is stated so that later increments can be checked against it.

3.3.1 Software Quality Attributes

- **Accuracy:** Each model shall be evaluated on held-out data not used for training or tuning, and shall report class-level precision, recall and F1-score in addition to accuracy, because overall accuracy is misleading on imbalanced security data [19]. Rare classes with too few test examples for a reliable estimate shall be identified as such.
- **Explainability:** Every alert or prediction shown to an analyst shall include the evidence behind it, such as confidence values or the most influential or most anomalous features.
- **Usability:** The planned dashboard shall let an analyst move from a summary of alerts to the details of a single alert in a few steps, using wording that does not require machine learning knowledge.
- **Maintainability:** Modules shall be independent, with model settings kept in configuration files, so that one module can be retrained or replaced without changing the others.
- **Compatibility:** The system shall run on Windows and Linux with a standard Python environment. The planned dashboard shall run in current web browsers.
- **Interoperability:** Module outputs shall use documented, structured formats such as JSON or CSV, so that they can be consumed by the planned backend or exported to other tools.
- **Availability:** In the demonstration environment, the integrated system shall remain usable if one module fails, reporting that module as unavailable instead of stopping completely.

- **Reproducibility:** Data splits, random seeds and configurations shall be fixed and saved, so that every reported result can be regenerated.

3.3.2 Performance Requirements

- **Detection quality.** On its benchmark test data, each supervised classifier shall reach a macro F1-score of at least 0.85. The current values are 0.958 for network threat classification and 0.877 for the malware category stage. The anomaly detector shall reach an F1-score of at least 0.80 at its calibrated threshold; the current mean values are 0.835 on CIC-IDS2017 and 0.899 on NSL-KDD.
- **False alarms.** The anomaly detector's false-positive rate shall be configurable through its threshold, and shall be reported on every evaluation. A target of at most 5% on traffic from the deployment network shall be used when recalibrating. The current test values are 2.9% on CIC-IDS2017 and 11.9% on NSL-KDD, so the latter needs recalibration.
- **Batch throughput.** Scoring a batch of 1,000 flow records with a trained model shall take less than one second on the development machine. The anomaly module currently takes about 35–52 milliseconds per 1,000 flows on a 12-core machine. Timings for the other modules will be measured during integration.
- **Training time.** Retraining a module on its benchmark dataset shall be possible on a standard development machine without specialised hardware.
- **Responsiveness.** The planned dashboard shall display an analysed batch of up to 10,000 records within a few seconds of the analysis completing.

3.3.3 Safety Requirements

- The system shall not execute blocking, isolation or other countermeasures automatically. Recommended responses shall be presented to an analyst for decision, because a wrong automatic action could disrupt legitimate services.
- The system shall be developed and tested only in a controlled environment, using public datasets and systems for which permission exists. It shall not be used to scan or attack third-party systems.
- The project shall not create, modify or execute malware. The malware module works only on numerical feature vectors from a public research dataset.
- When a module receives input it cannot process correctly (for example, missing feature columns), it shall report an error or warning instead of returning results that look normal (see FR-M2 and FR-A8).
- The limitations of each model, such as benchmark-only evaluation and weak results on rare classes, shall be shown alongside its results so that its outputs are not over-trusted.

3.3.4 Other Non-Functional Requirements

- **Security of the platform:** The planned backend and dashboard shall require authentication, restrict stored security data by role, and protect it in storage and in transit.

- **Privacy:** Direct identifiers such as IP addresses shall not be used as model inputs. They may appear only in alerts, as context for the analyst.
- **Legal and ethical use:** Datasets shall be used according to their published terms, and all sources shall be cited.
- **Portability:** The trained models and their preprocessing objects shall be stored together, so that a module can be moved to another machine and loaded without retraining.

3.4 Requirements Gathering Techniques Used

A requirements gathering technique is a method for discovering and analysing what stakeholders need from a system, such as interviews, surveys, workshops, observation, document analysis or prototyping [36]. The choice of techniques depends on the project. This project is a research prototype without an external client, and its users (security analysts) were not directly accessible to the team, so requirements were drawn mainly from published knowledge, the data itself and expert guidance. Three techniques were used. Interviews and surveys with practitioners were not carried out, and are a candidate technique for validating the dashboard requirements in the next increment.

3.4.1 Literature and Existing System Review

The team studied published research on machine learning for intrusion detection, anomaly detection and malware analysis [6], [7], [20], together with critical studies of how such systems are evaluated [8], [9]. The team also reviewed the documentation of existing security platforms (Section 2.1). This review shaped several requirements. The need to cover unseen attacks led to the anomaly module (FR-A1 to FR-A4). The known weaknesses of accuracy-only evaluation led to the accuracy and explainability attributes. The way commercial platforms combine evidence from several sources informed the integration requirements (FR-S1 to FR-S7).

3.4.2 Supervisor Meetings

Regular meetings with the project supervisor were used to agree the project scope, to divide the work into modules, and to review results as each module was developed. These meetings set the boundaries in Section 1.2, such as working in a controlled environment, making recommendations instead of automatic responses, and not handling live malware. They also determined the order of the increments and confirmed that integration should follow once the first three modules were complete.

3.4.3 Dataset and Document Analysis

The public datasets and their documentation were analysed in detail before and during development: CIC-IDS2017 [14], [35], NSL-KDD [15] and BODMAS [16]. This analysis defined the input formats of the modules (FR-N1, FR-M1), the class taxonomies (FR-N4, FR-M4) and several quality requirements. For example, examining CIC-IDS2017 revealed missing and infinite values, extreme class imbalance and a large number of duplicate flows. This led to the requirements on preprocessing, class-level metrics and leakage-free evaluation. Reference documents were also used: the MITRE ATT&CK knowledge base for attack techniques [18] and NIST incident response guidance [1] for

the steps an investigation should support.

3.5 Time Frame

A schedule helps a team divide a large project into manageable phases, allocate its time realistically and notice early when work is falling behind. In this project, the schedule follows the increments described in Section 2.5. Each phase ends with a deliverable that is reviewed with the supervisor before the next phase begins. Table 3.2 summarises the completed phases and the planned schedule for the rest of the project. Durations for future phases are estimates and will be revised as the work progresses.

Table 3.2: Time Frame

Phase	**Main deliverables**	**Duration**	**Status**
Phase 1: Proposal and requirements	Long proposal, module division, initial literature review	4 weeks	Completed
Phase 2: Increment 1 (analysis modules)	Network threat, malware and anomaly modules trained and evaluated	6 weeks	Completed
Phase 3: Mid-term report	Chapters 1–3, mid-term presentation	2 weeks	In progress
Phase 4: Increment 2 (log analysis and integration)	Log analysis module, backend API, database, common result format	6 weeks	Planned
Phase 5: Increment 3 (analysis extensions)	Incident correlation, risk, vulnerability and attack-path analysis, investigation agent	6 weeks	Planned
Phase 6: Increment 4 (dashboard and testing)	Dashboard, analyst feedback, integration and system testing	4 weeks	Planned
Phase 7: Final report and demonstration	Chapters 4–7, final presentation	2 weeks	Planned

References

[1] A. Nelson, S. Rekhi, M. Souppaya, and K. Scarfone, "Incident response recommendations and considerations for cybersecurity risk management: A CSF 2.0 community profile," National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST Special Publication 800-61 Rev. 3, Apr. 2025. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-61r3

[2] Splunk Inc., "Splunk enterprise security," https://www.splunk.com/en_us/products/enterprise-security.html, accessed: Nov. 6, 2026.

[3] Microsoft, "What is Microsoft Sentinel SIEM?" Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/azure/sentinel/overview, accessed: Nov. 6, 2026.

[4] CrowdStrike, "The CrowdStrike Falcon platform," https://www.crowdstrike.com/en-us/platform/, accessed: Nov. 6, 2026.

[5] Wazuh, Inc., "Capabilities – Wazuh documentation," https://documentation.wazuh.com/current/user-manual/capabilities/index.html, accessed: Nov. 6, 2026.

[6] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016. [Online]. Available: https://doi.org/10.1109/COMST.2015.2494502

[7] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 15:1–15:58, Jul. 2009. [Online]. Available: https://doi.org/10.1145/1541880.1541882

[8] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Security and Privacy (S&P)*, 2010, pp. 305–316. [Online]. Available: https://doi.org/10.1109/SP.2010.25

[9] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symp.*, Boston, MA, USA, 2022, pp. 3971–3988. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/arp

[10] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html

[11] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf

[12] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, vol. 31. Curran Associates, Inc., 2018. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2018/file/14491b756b3a51daac41c24863285549-Paper.pdf

[13] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan, and J. Zou, "Gradio: Hassle-free sharing and testing of ML models in the wild," arXiv preprint arXiv:1906.02569, 2019. [Online]. Available: https://arxiv.org/abs/1906.02569

[14] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*. SciTePress, 2018, pp. 108–116. [Online]. Available: https://doi.org/10.5220/0006639801080116

[15] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Computational Intelligence for Security and Defense Applications (CISDA)*, 2009, pp. 1–6. [Online]. Available: https://doi.org/10.1109/CISDA.2009.5356528

[16] L. Yang, A. Ciptadi, I. Laziuk, A. Ahmadzadeh, and G. Wang, "BODMAS: An open dataset for learning based temporal analysis of PE malware," in *Proc. IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 78–84. [Online]. Available: https://doi.org/10.1109/SPW53761.2021.00020

[17] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Military Communications and Information Systems Conf. (MilCIS)*, 2015, pp. 1–6. [Online]. Available: https://doi.org/10.1109/MilCIS.2015.7348942

[18] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and philosophy," The MITRE Corporation, McLean, VA, USA, Tech. Rep. MP180360, Jul. 2018. [Online]. Available: https://www.mitre.org/news-insights/publication/mitre-attck-design-and-philosophy

[19] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, p. e0118432, 2015. [Online]. Available: https://doi.org/10.1371/journal.pone.0118432

[20] D. Ucci, L. Aniello, and R. Baldoni, "Survey of machine learning techniques for malware analysis," *Computers & Security*, vol. 81, pp. 123–147, Mar. 2019. [Online]. Available: https://doi.org/10.1016/j.cose.2018.11.001

[21] M. Ahmed, A. N. Mahmood, and J. Hu, "A survey of network anomaly detection techniques," *Journal of Network and Computer Applications*, vol. 60, pp. 19–31, Jan. 2016. [Online]. Available: https://doi.org/10.1016/j.jnca.2015.11.016

[22] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422. [Online]. Available: https://doi.org/10.1109/ICDM.2008.17

[23] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2000, pp. 93–104. [Online]. Available: https://doi.org/10.1145/342009.335388

[24] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504–507, 2006. [Online]. Available: https://doi.org/10.1126/science.1127647

[25] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Network and Distributed System Security Symp. (NDSS)*, San Diego, CA, USA, 2018. [Online]. Available: https://doi.org/10.14722/ndss.2018.23204

[26] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. [Online]. Available: https://doi.org/10.1023/A:1010933404324

[27] H. S. Anderson and P. Roth, "EMBER: An open dataset for training static PE malware machine learning models," arXiv preprint arXiv:1804.04637, 2018. [Online]. Available: https://arxiv.org/abs/1804.04637

[28] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: the CICIDS2017 case study," in *Proc. IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12. [Online]. Available: https://doi.org/10.1109/SPW53761.2021.00009

[29] Elastic N.V., "Elastic security – Elastic docs," https://www.elastic.co/docs/solutions/security, accessed: Nov. 6, 2026.

[30] Security Onion Solutions, LLC, "Security onion documentation," https://docs.securityonion.net/, accessed: Nov. 6, 2026.

[31] M. Roesch, "Snort – lightweight intrusion detection for networks," in *Proc. 13th USENIX Conf. System Administration (LISA '99)*, 1999, pp. 229–238. [Online]. Available: https://www.usenix.org/legacy/event/lisa99/roesch.html

[32] Open Information Security Foundation, "Suricata," https://suricata.io/, accessed: Nov. 6, 2026.

[33] V. Paxson, "Bro: A system for detecting network intruders in real-time," *Computer Networks*, vol. 31, no. 23–24, pp. 2435–2463, Dec. 1999. [Online]. Available: https://doi.org/10.1016/S1389-1286(99)00112-7

[34] The Zeek Project, "The Zeek network security monitor," https://zeek.org/, accessed: Nov. 6, 2026.

[35] Canadian Institute for Cybersecurity, "Intrusion detection evaluation dataset (CIC-IDS2017)," University of New Brunswick. [Online]. Available: https://www.unb.ca/cic/datasets/ids-2017.html, accessed: Nov. 6, 2026.

[36] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.

[37] C. Larman and V. R. Basili, "Iterative and incremental development: A brief history," *Computer*, vol. 36, no. 6, pp. 47–56, Jun. 2003. [Online]. Available: https://doi.org/10.1109/MC.2003.1204375

[38] ISO/IEC/IEEE, "ISO/IEC/IEEE 29148:2018 – systems and software engineering – life cycle processes – requirements engineering," International Standard, 2nd ed., Nov. 2018. [Online]. Available: https://doi.org/10.1109/IEEESTD.2018.8559686